Week 11: Test, polish and present

A unit test

```
import unittest
from grades import grade

class TestGrade(unittest.TestCase):
    def test_boundaries(self):
        self.assertEqual(grade(75), "A")
        self.assertEqual(grade(74), "B")

    def test_bad_input(self):
        with self.assertRaises(ValueError):
            grade(101)

if __name__ == "__main__":
    unittest.main()
```

```
python3 -m unittest -v
python3 -m unittest discover -s tests -v
```

Assertion	Checks
`assertEqual(a, b)`	`a == b`
`assertTrue(x)` / `assertFalse(x)`	Truth
`assertIn(a, b)` / `assertNotIn`	`a` is (not) inside `b`
`assertIsNone(x)`	`x is None`
`assertRaises(Err)`	Code raises that error

Arrange (set up) → **Act** (call it) → **Assert** (check). `setUp` runs before **every** test. `with self.subTest(x=x):` reports each failing value.

Test a Flask app

```
def setUp(self):
    self.tmp = tempfile.TemporaryDirectory()
    app.config.update(
        DATABASE=self.tmp.name + "/t.db",
        TESTING=True)
    with app.app_context():
        init_db(seed=True)          # fresh data
    self.client = app.test_client()

r = self.client.post("/new", data={"name": "x"})
self.assertEqual(r.status_code, 302)   # redirect
html = self.client.get("/").get_data(as_text=True)
```

- Each test gets its **own temporary database**; never test on real data.
- Check the **database** (`SELECT`) as well as the page.
- `follow_redirects=True` follows the redirect to the page after it.

What to test

Kind	Example
Happy path	Valid form saves a row
Invalid input	Rejected; message shown; nothing saved
Boundary	Largest valid value passes; one more fails
Not found	Unknown id → 404
Wrong method	GET on a POST route → 405
Database rule	Constraint refuses a bad row
Hostile input	`' OR 1=1 --`, `<script>` are harmless

Bugs hide on **boundaries**. Test the value on each side of every limit. A passing test shows only that the cases you thought of work.

Polish checklist

- Wording consistent; no template text, debug prints or "Test 1" data left.
- Every list has an empty state message.
- Error messages are beside the field and say how to fix it.
- Labels on all inputs; 16px text; 44px buttons; works at iPad width.
- Colour is never the only signal; headings in order; `alt` text.
- Dangerous actions are POST buttons that look different.

README sections

Problem · Features · How to run · ER diagram and design · Design decisions · Testing · Limitations and next steps · What I learned.

On a real server

- **No debug mode.** No `app.run()` at the top level.
- Secret key from the environment: `os.environ.get("SECRET_KEY")`.
- Build the database with `flask --app app init-db` **once**; it is not in Git.
- Deploy steps: clone → virtualenv → `init-db` → WSGI file → Reload.

Present: 3 minutes

Part	Time
The problem	30 s
Live demo of one story, with an error	90 s
ER diagram, one query, one test	40 s
Reflection: hardest bug, next step	20 s